BookMyShow Ticketing System

Database Design, Showtime Query & Concurrency Safeguards

Airtribe Institute AI Learning Project

Author: Udhaya Sankar

Database: MySQL 8.0

Project summary

This project models a movie-ticket booking database for a theatre/date/showtime workflow. It documents core entities, normalized relational design, sample-data expectations, the P2 showtime query, and database safeguards that form the foundation for seat holds and idempotent payments.

Implementation status

The SQL schema and sample queries have been exercised in MySQL Workbench. A duplicate booking-seat insert was rejected with MySQL error 1062, confirming the unique seat/show constraint. Redis locking, automatic hold release, full payment webhook processing, and a high-concurrency load test must be implemented and tested separately before claiming end-to-end validation.

1. Scenario and Requirements

For a selected theatre, the application presents the next seven dates. When a date is selected, users see movies and showtimes at that theatre on that date. The database must preserve booking integrity when multiple users compete for the same seats.

Core requirements

• Store users, theatres, screens, movies, physical seats, shows, bookings, booked seats, payments, webhook events, and temporary seat holds.
• Retrieve shows for a theatre and date, ordered by show start time.
• Enforce referential integrity and prevent duplicate booking-seat records.
• Record hold expiry and webhook event identity so application services can release holds and deduplicate payment events.

2. Entities and Attributes

users

Attribute	Type	Constraint / meaning
user_id	BIGINT UNSIGNED	PK, auto-increment
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
phone	VARCHAR(20)	Optional
created_at	TIMESTAMP	Default current timestamp

theatres

Attribute	Type	Constraint / meaning
theatre_id	BIGINT UNSIGNED	PK, auto-increment
theatre_name	VARCHAR(150)	NOT NULL
city	VARCHAR(100)	NOT NULL
address	VARCHAR(255)	NOT NULL

screens

Attribute	Type	Constraint / meaning
screen_id	BIGINT UNSIGNED	PK, auto-increment
theatre_id	BIGINT UNSIGNED	FK → theatres
screen_name	VARCHAR(100)	NOT NULL; unique within theatre

movies

Attribute	Type	Constraint / meaning
movie_id	BIGINT UNSIGNED	PK, auto-increment
title	VARCHAR(200)	NOT NULL
language	VARCHAR(50)	NOT NULL
duration_minutes	SMALLINT UNSIGNED	NOT NULL
certificate	VARCHAR(20)	Optional
release_date	DATE	Optional

seats

Attribute	Type	Constraint / meaning
seat_id	BIGINT UNSIGNED	PK, auto-increment
screen_id	BIGINT UNSIGNED	FK → screens
row_label	VARCHAR(10)	NOT NULL
seat_number	SMALLINT UNSIGNED	NOT NULL
seat_type	ENUM	REGULAR / PREMIUM / RECLINER; position unique per screen

shows

Attribute	Type	Constraint / meaning
show_id	BIGINT UNSIGNED	PK, auto-increment
movie_id	BIGINT UNSIGNED	FK → movies
screen_id	BIGINT UNSIGNED	FK → screens
show_date	DATE	NOT NULL
start_time	TIME	NOT NULL
end_time	TIME	NOT NULL
language	VARCHAR(50)	NOT NULL
ticket_price	DECIMAL(10,2)	NOT NULL

bookings

Attribute	Type	Constraint / meaning
booking_id	BIGINT UNSIGNED	PK, auto-increment
user_id	BIGINT UNSIGNED	FK → users
show_id	BIGINT UNSIGNED	FK → shows
booking_reference	VARCHAR(50)	UNIQUE
status	ENUM	PENDING / CONFIRMED / CANCELLED / EXPIRED
total_amount	DECIMAL(10,2)	NOT NULL
created_at	TIMESTAMP	Default current timestamp
expires_at	TIMESTAMP	Optional

booking_seats

Attribute	Type	Constraint / meaning
booking_seat_id	BIGINT UNSIGNED	PK, auto-increment
booking_id	BIGINT UNSIGNED	FK → bookings
show_id	BIGINT UNSIGNED	FK → shows
seat_id	BIGINT UNSIGNED	FK → seats
price	DECIMAL(10,2)	NOT NULL; UNIQUE(show_id, seat_id)

payments

Attribute	Type	Constraint / meaning
payment_id	BIGINT UNSIGNED	PK, auto-increment
booking_id	BIGINT UNSIGNED	FK → bookings
provider	VARCHAR(50)	NOT NULL
provider_payment_id	VARCHAR(150)	Unique with provider
amount	DECIMAL(10,2)	NOT NULL
status	ENUM	PENDING / SUCCESS / FAILED / REFUNDED

payment_webhook_events

Attribute	Type	Constraint / meaning
webhook_event_id	BIGINT UNSIGNED	PK, auto-increment
provider	VARCHAR(50)	NOT NULL
event_id	VARCHAR(150)	Unique with provider
event_type	VARCHAR(100)	NOT NULL
payload	JSON	NOT NULL
processed	BOOLEAN	Default FALSE
received_at	TIMESTAMP	Default current timestamp

seat_holds

Attribute	Type	Constraint / meaning
hold_id	BIGINT UNSIGNED	PK, auto-increment
show_id	BIGINT UNSIGNED	FK → shows
seat_id	BIGINT UNSIGNED	FK → seats
user_id	BIGINT UNSIGNED	FK → users
hold_token	VARCHAR(100)	UNIQUE
status	ENUM	ACTIVE / RELEASED / CONVERTED / EXPIRED
expires_at	TIMESTAMP	NOT NULL
created_at	TIMESTAMP	Default current timestamp

3. Sample Data

Example theatre row (illustrative; actual IDs depend on the seed script):

theatre_id	theatre_name	city
1	PVR Cinemas	Chennai

Example booking-seat records can associate booking 1 with seats 1 and 2 for show 1 at ■180.00 each. Treat these as illustrative; use actual rows produced by sql/03_insert_sample_data.sql for reported counts.

4. P1 — Normalization and Integrity

1NF: Each column contains one atomic value; repeating groups are represented as separate rows.

2NF: Non-key attributes depend on the whole candidate key. Primary keys and unique business keys express record identity.

3NF: User, theatre, screen, movie, booking, and payment attributes are separated to reduce duplication and update anomalies.

BCNF: Every non-trivial functional dependency X → Y must have X as a superkey. Validate each table's candidate keys and functional dependencies; table separation alone does not prove BCNF.

Key constraints

• UNIQUE (show_id, seat_id) on booking_seats prevents duplicate booked-seat rows for a show.
• UNIQUE (provider, event_id) on payment_webhook_events supports webhook deduplication.
• Foreign keys maintain relationships between entities.
• Price and amount checks should reject negative values where included in the schema.

5. P2 — Shows for a Theatre and Date

The query joins shows to screens, theatres, and movies, filters by theatre and date, and orders results by start time. Use the exact column names from sql/05_p2_queries.sql.

```
SELECT t.theatre_name, t.city, m.title AS movie_title,
m.language AS movie_language, sc.screen_name, s.show_date,
s.start_time, s.end_time, s.ticket_price
FROM shows AS s
JOIN screens AS sc ON sc.screen_id = s.screen_id
JOIN theatres AS t ON t.theatre_id = sc.theatre_id
JOIN movies AS m ON m.movie_id = s.movie_id
WHERE t.theatre_id = 1 AND s.show_date = '2026-10-06'
ORDER BY s.start_time;
```

Expected result: one row per show scheduled at the selected theatre and date. The row count depends on the loaded sample data.

6. Concurrency Strategy

Database uniqueness: UNIQUE (show_id, seat_id) is the final guard against duplicate booking-seat rows. The manual MySQL test rejected a duplicate insert with error 1062.

Timed holds: Redis can provide atomic short-lived seat holds with TTL. The service must validate hold ownership and expiry before confirming a booking. A worker should reconcile expired persistent hold records.

Transactions: Confirm a multi-seat booking in a MySQL transaction using suitable row locks or conditional updates. Roll back if any requested seat cannot be acquired.

Webhook idempotency: Store a unique provider/event key and update payment/booking state transactionally. Verify signatures, amount, and payment status before confirmation.

Queues: Queues can absorb bursts for notifications and asynchronous work, while seat allocation still requires atomic locking and database integrity.

7. Indexing and Testing

Index design should follow actual query patterns: show/date/time lookups, booking lookup by user/status, booking seats by show/seat, hold expiry by status/expiry, and webhook event identity. Use EXPLAIN and measured workload to tune indexes.

SQL scripts

• 01_create_database.sql — create/select database.
• 02_create_tables.sql — table definitions and constraints.
• 03_insert_sample_data.sql — demonstration data.
• 04_p1_queries.sql — P1 verification.
• 05_p2_queries.sql — P2 showtime query.
• 06_concurrency_tests.sql — integrity and hold checks.

Testing status

The duplicate-seat guard was manually verified in MySQL Workbench. A zero-row duplicate query means no duplicates were present in the inspected data. A zero-row expired-hold query means no expired active holds matched at that moment; it does not prove automatic release. Redis locking, webhook handling, and load testing require separate end-to-end tests.

8. Conclusion

This project documents a relational foundation for theatre/date/showtime ticketing and the safeguards needed for seat integrity and payment-event deduplication. Production-scale claims require implementation and measurement of application locking, timed hold release, payment processing, and concurrent load tests.